DATA-260

HOMEWORK 4

Instructions

- Please provide the code snippet for each question and sub questions along with its intended output.
  Ensure that the code and corresponding output screenshots are placed together, one below the other.
- Work is submitted through your GitHub repository **data260-<SID4>** (the same repository from HW1, extended further).
- Submit **report.pdf** (name your submitted PDF as **{last_name}_HW4.pdf**) with your GitHub repository link included inside it, and confirm the link is working and that both collaborators (**Sbnikitha**, **supriyaselvanganesan**) have access.
- Keep all application code in shared root-level folders and continue extending your existing codebase from previous homeworks. Do not create a separate copy of the code for this homework.

Section 0 - Personal Configuration and Domain

Before starting, compute your personal configuration values from your SJSU Student ID and record them at the top of your report. These values stay fixed for the rest of the semester.

Value	Definition
SID4	Last four digits of your SJSU Student ID
PORT_BASE	8000 + (SID4 mod 900) - the port your app must run on
PREFIX	"s" + SID4
SEED	SID4
VERIFY_SEED	260000 + SID4
DOMAIN_ID	SID4 mod 8 - determines your assigned domain

At the top of your report, also state your hardware, local model used, and the tagged commit hash for this submission, in addition to the six configuration values above.

Repository Instruction

In your repository, create **reports/hw04/** containing:

- **RUN_LOG.txt** - real console output and timestamps for the runs that produced your reported results.

- **raw/** - the machine-readable CSV/JSON files from your experiments (all 180 measured N+1 requests: 3 page sizes × 2 implementations × 30 requests each).
- **METRICS.md** - your filled-in results tables.
- **AI_USE.md** - answers to four questions: (1) what you used an AI assistant for and what you did yourself; (2) one AI-produced output that was wrong/unsuitable, or one thing you independently verified;

  (3) how you detected the problem or verified the result; (4) what you changed and why it works now.
- **report.pdf** - your write-up with screenshots and answers.

verification.json - output of a self-check script (e.g. make verify-hw04). Write a small script that runs a smoke test on your tagged commit and writes this file. A smoke test just means: start up your system and check that the basic things work, not every edge case. For example: does the FastAPI backend respond on PORT_BASE? Does the naive vs. fixed list endpoint both return data? Your checks should be objective - testing behavior, not exact wording. It should contain:

- Homework number
- SID4
- Commit hash
- Model/configuration used (when applicable)
- SEED / VERIFY_SEED
- The checks that were run
- Pass/fail for each check

Your script can create temporary test files while it runs, but it must not modify your actual application code.

Tag your submitted repository state **hw4**.

Part 1 - React Client

This homework builds on HW3. Your existing FastAPI backend becomes the backend for the React client below, and your domain data moves to MySQL. Keep extending your existing service - don't start a new one.

Create a domain application where users can add, update, and delete records for your assigned domain entity.

Add a Login page/component (**Login.jsx**) that allows a user to log in using email + password.

- After successful login, the backend will set an HTTP-only cookie containing a server-side session token.
- Only logged-in users should be allowed to view the record list and perform CRUD operations.
- If the user is not logged in: show a message "Login required", and/or disable/hide navigation buttons like "Add Record".

I. Home Page

Write the necessary code to create the **Home.jsx** component which will display all the records. The Home component should be rendered when the user is on the root route /.

II. Add a New Record

Write the necessary code to create the **CreateRecord.jsx** component. The component should be rendered when the user is on the /create route.

The component should accept props to add the new record and have the following:

- An input field for entering the entity's primary field.
- An input field for entering the entity's secondary field.
- A submit button labeled appropriately for your domain.
- When the user clicks the submit button:
- A new record should be added to the list with an auto-incremented ID.
- The user should be redirected to the home page to see the updated list.

III. Update Record

Write the necessary code to create the **UpdateRecord.jsx** component. The component should be rendered when the user is on the /update route.

The component should accept props to update the record and have the following:

- An input field for the primary field.
- An input field for the secondary field.
- A submit button labeled appropriately for your domain.

When the user clicks the submit button, the record should be updated through the backend API and persisted in MySQL and the user should be redirected to the home page showing the updated list.

IV. Delete the Record

Write the necessary code to create the **DeleteRecord.jsx** component. The component should be rendered when the user is on the /delete route.

The component should accept props to delete a record and have the following:

- A button labeled appropriately for your domain (e.g., "Delete").

When the user clicks the delete button, the record should be deleted through the backend API and removed from MySQL and the user should be redirected to the home page showing the updated list.

V. For all the above components, make sure to:

- Pass props for the Create, Update, and Delete components.
- Use hooks like useState and useEffect wherever necessary.
- Use React Router (react-router-dom) for routing.

Part 2 - MySQL Persistence and Server-Side Sessions

In this part, you will apply the concepts learned in class to build a simple CRUD (Create, Read, Update, Delete) application using FastAPI and MySQL, using SQLAlchemy.

Database

Create MySQL database: <PREFIX>_rel
Table: your primary domain entity (auto-increment id, primary field, secondary field)

Also create tables for authentication/session management:

- users (id, name, email (unique), password_hash)
- sessions (id (session token), user_id, created_at, expires_at)

You are required to create a CRUD system for your domain entity that allows users to:

- Add a new record (POST)
- View all records (GET)
- View a record by ID (GET)
- Update record details (PUT)
- Delete a record (DELETE)

You are free to name your API endpoints appropriately.

The session token issued at login must be stored server-side in the sessions table and referenced only by an opaque token in the browser's HTTP-only cookie - the cookie itself should never store user data directly.

Additionally submit Postman API response screenshots for each operation. Also submit a screenshot of your database and your project folder structure.

Part 3 - N+1 Measurement and Query Tuning

1. Seed 5,000 rows of your primary domain entity and 200 related rows, using your SEED value. This related table is just test data for now - you'll build a proper related entity with full CRUD in a later homework. Commit your generator script and the schema/migration you used to create these rows.
2. The 200 related rows must be associated with the primary domain entity, and the list endpoint's response must include this related data.
3. Build a list endpoint that runs one extra query per record to fetch its related data (this is the classic "N+1" problem - intentionally naive for now).
4. Test this endpoint at page sizes 10, 50, and 200. For each size, run 30 requests and record how many SQL queries each request triggers, plus the p50/p95/p99 latency.

5 Now fix the N+1 issue using a join or eager loading, and re-run the same measurements.

6 Save the raw data for all 180 requests (3 page sizes × 2 versions × 30 requests) in reports/hw04/raw/, so your results can be double-checked.

7 Report how much faster the fixed version is at each page size, and explain why the speed-up changes as the page size grows.

8 Add one index to your database, and show the EXPLAIN output before and after adding it - explain what changed.

9 Include Postman screenshots showing both versions of the endpoint working correctly at each page size.

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive				
10	fixed				
50	naive				
50	fixed				
200	naive				
200	fixed				

Part 4 - Build a Grounded RAG Question-Answering System

Build a small Retrieval-Augmented Generation (RAG) application that answers questions using only a supplied document collection and use it to study context engineering. Prompt engineering is how you ask the model; context engineering is what evidence you place in the model's window - the retrieved chunks, their ordering and de-duplication, and the token budget.

1. **Corpus and index:** use at least 5 documents (course notes, policies, or technical docs). Chunk them (start with chunk_size = 500, chunk_overlap = 50), storing each chunk's text, source name, and chunk_id; embed the chunks and load them into a vector store (FAISS, Chroma, or LlamaIndex VectorStoreIndex).
2. **Retrieval:** for each question, retrieve the top-k chunks (start with top_k = 3) and print the retrieved chunks with source and score before calling the LLM, so a retrieval problem is visible separately from a generation problem.
3. **Run the same questions through three configurations:**
    - **(A) No RAG** - question straight to the LLM, as a baseline.
    - **(B) Basic RAG** - top-3 raw chunks in the prompt.

- **(C) Context-engineered RAG** - drop irrelevant and duplicate chunks, order and label the survivors with their sources, and add grounding rules: answer only from the context, cite the source number, and refuse with "I cannot answer this question from the provided documents" when the evidence is insufficient.

4. **Test set of six questions:**
   - Q1: answer in one chunk
   - Q2: answer needs two chunks
   - Q3: similar information across documents
   - Q4: ambiguous
   - Q5: answer not in the documents
   - Q6: unrelated

   Q5 and Q6 must be refused, not answered.

5. **Sweep the context size:** run at least one question at k = 1, 3, and 5, and report whether more context helped, when irrelevant chunks appeared, and which k was best - more context is not always better context.
6. **Evaluate:** for each question and configuration, record correct retrieval, correct answer, grounded (answer supported by the cited chunks), and refused-when-needed; summarize accuracy, faithfulness, format compliance, and robustness in one table.
7. **Write a short analysis (300–500 words):** which questions retrieved the right chunks, which context changes helped (top_k, de-duplication, chunk size, source metadata, ordering, shorter context, grounding rules), whether the model hallucinated on Q5/Q6, and how retrieval quality, context quality, and the prompt together shaped answer quality.

Save your retrieved-chunk printouts, the three-configuration comparison, the k-sweep results, and the evaluation table in reports/hw04/raw/, and include your written analysis in report.pdf.

Required Deliverables

rag.py and the document corpus; printed top-k retrievals; the No-RAG / Basic-RAG / Context-RAG comparison; the six-question results including refusals on Q5/Q6; the top_k sweep; the evaluation table; and the short analysis.

Include the relevant code, outputs, tables, and analysis in your submission.

Make one PDF that includes: your code snippet and output screenshot for every question and sub-question asked in this homework, add your written analysis where required, and your AI_USE.md answers. Every part of every question needs its own screenshot.